Chapter 11 — Lambdas and Functional Programming

Before Java 8, handing a piece of *behaviour* to a method — "sort by length", "run this later", "keep only the big orders" — meant wrapping it in an object, usually an anonymous class with five lines of ceremony around one line of logic. **Lambdas** (Java 8) let you write that behaviour directly as a value: `s -> s.length()`. With **functional interfaces**, **method references** and `Optional` they brought a **functional style** (programming by passing and combining functions rather than changing variables step by step) that collections (Chapter 10) and streams (Chapter 12) are built on.

All runnable code is in `code/` (package `ch11.*`), compiled with `javac --release 25` (target the Java 25 language and library level) and run on JDK 26.

> **Exam depth.** Which lambda forms compile, `var` in parameters, method-reference puzzles, the full built-in interface table with method names (`test`, `apply`, `accept`, `get`, `applyAsInt` …) and "effectively final" traps are drilled in OCP Chapter 8 — Lambdas and functional interfaces (see *Gotchas & exam traps*). `Optional` exam questions are in OCP Chapter 10.

11.1 Why lambdas: passing behaviour

The problem it solves

Many methods need a small piece of logic from the caller: `sort` needs "how to compare", a thread needs "what to run", a filter needs "what to keep". Making behaviour an argument, just like data, is **behaviour parameterisation**. Before Java 8 the only way was an **anonymous class** (defined and instantiated in one expression, §6.11):

```
names.sort(new Comparator<String>() {
    @Override public int compare(String a, String b) { return Integer.compare(a.length(), b.length()); }
});
```

Five lines, one idea. A **lambda expression** is an anonymous function — parameters, an arrow, a body — and the receiving method decides when (and whether) to run it:

```
names.sort((a, b) -> Integer.compare(a.length(), b.length()));
```

Why it matters

- **Less noise, more intent** — *Effective Java* (Joshua Bloch's book of numbered best-practice "Items") Item 42: "prefer lambdas to anonymous classes".
- Function-taking APIs: `Map.merge`, `List.removeIf`, `Comparator.comparing`, `CompletableFuture`, Streams.
- Deferred and conditional execution: pass a `Supplier` and the work happens only if needed (§11.8).

11.2 Functional interfaces and lambda syntax

What it is

A **functional interface** has **exactly one abstract method** (a "single abstract method", or **SAM**, type); a lambda is a compact way to create an instance of one. Classic examples: `Runnable` (`void run()`), `Comparator<T>` (`int compare(T, T)`), `Callable<V>`.

Rules:

1. Only *abstract* methods count. `default` and `static` methods are fine, and so are abstract re-declarations of `Object`'s public methods (that's why `Comparator` can declare `equals` and still be functional).

2. The optional `@FunctionalInterface` annotation makes the compiler verify the "exactly one" rule — a guard against someone later adding a second abstract method.
3. A lambda has **no type of its own**. The compiler infers it (**type inference**) from the **target type** of its context (assignment, method parameter, return, cast), so `s -> s.isEmpty()` can become a `Predicate<String>` or a `Function<String, Boolean>`. That's why `var f = x -> x;` (or `var f = () -> 42;`) doesn't compile: there is no target type.

Syntax

Form	Example	Notes
No parameters	`() -> System.out.println("hi")`	parentheses required
One parameter, inferred type	`s -> s.length()`	parentheses optional
Several parameters	`(a, b) -> a + b`	parentheses required
Explicit types	`(Integer a, Integer b) -> a + b`	all or none typed
`var` parameters (Java 11)	`(var a, var b) -> a * b`	lets you add annotations: `(@NonNull var s) -> …`; all or none
Unnamed parameter (Java 22)	`(key, _) -> key`	`_` for a parameter you don't use
Expression body	`x -> x * 2`	value of the expression is returned
Block body	`n -> { if (n < 0) return "neg"; return "ok"; }`	braces, semicolons and `return` required

Code

```java
// code/ch11/lambdas/LambdaBasics.java (key parts)
@FunctionalInterface
interface Discount {
    double apply(double price);                                    // the single abstract method
    default Discount then(Discount next) { return p -> next.apply(apply(p)); }   // defaults allowed
}

Runnable noArgs                 = () -> System.out.println("no parameters");
Function<String, Integer> oneArg = s -> s.length();
BinaryOperator<Integer> typed    = (Integer a, Integer b) -> a + b;
BiFunction<Integer, Integer, Integer> withVar = (var a, var b) -> a * b;
Function<Integer, String> block  = n -> { if (n < 0) return "negative"; return "non-negative"; };

Predicate<String> isEmpty1          = s -> s.isEmpty();   // same text,
Function<String, Boolean> isEmpty2  = s -> s.isEmpty();   // different target types

Discount tenOff = p -> p - 10, twentyPercent = p -> p * 0.8;
tenOff.then(twentyPercent).apply(100);                     // (100 - 10) * 0.8

Map<String, BinaryOperator<Integer>> ops = Map.of("+", Integer::sum, "*", (a, b) -> a * b, "max", Math::max);
```

Output:

```
anonymous class: [al, Bea, Charlie]
lambda:          [Charlie, Bea, al]
no parameters
6 5 20 negative
Predicate.test=true Function.apply=false
100 -> tenOff then 20% = 72.0
+=13 *=42 max=7
```

The last line shows lambdas are *values*: stored in a map, looked up, called.

How it works under the hood

1. The compiler turns the lambda body into a private method of the enclosing class (named like `lambda$main$0`).
2. At the lambda's location it emits an **`invokedynamic`** bytecode instruction (§1.6), whose target is decided at run time on first use. That first time, the JVM's `LambdaMetafactory` generates a small class implementing the functional interface that calls the private method.
3. No `.class` file per lambda appears on disk (anonymous classes produce `Outer$1.class`).
4. A non-capturing lambda is typically created once and reused; a capturing one gets a new instance holding the values. (Identity is unspecified either way — don't rely on it.)

Lambda vs anonymous class

	Lambda	Anonymous class
Implements	only functional interfaces	any interface or class (can extend an abstract class)
Number of methods	one	any; can have fields and state
`this` refers to	the **enclosing** instance	the anonymous object itself
Shadowing (redeclaring an outer name)	can't redeclare a local variable of the enclosing scope	can shadow
Compiled as	`invokedynamic` + private method	a separate class file

Use an anonymous class only when you need state, several methods, or to extend a class.

Common mistakes

- A block body that forgets `return`, or an expression body followed by a semicolon inside: `x -> { x * 2 }` ✗ vs `x -> x * 2` ✓.
- Mixing typed and untyped parameters: `(String a, b) -> …` ✗.
- Naming a lambda parameter the same as an existing local variable → "variable is already defined".
- Long multi-statement lambdas — extract a named method and use a method reference; stack traces and readers both benefit.

TL;DR

A lambda is an anonymous function that implements a functional interface (one abstract method). Its type comes from context. Syntax: `params -> expression` or `params -> { statements; }`. Prefer it over anonymous classes unless you need state or `this` to mean the object itself.

11.3 The built-in functional interfaces (`java.util.function`)

`java.util.function` has **43** interfaces covering the common shapes, so you rarely write your own. *Effective Java* Item 44: favour the standard ones — callers know them and they come with useful **default methods** (interface methods with a body).

The core shapes

Interface	Shape	Method	Typical use
`Supplier<T>`	`() → T`	`get()`	factories, lazy values, defaults
`Consumer<T>`	`T → void`	`accept(t)`	`forEach`, logging, sending
`BiConsumer<T,U>`	`(T, U) → void`	`accept(t, u)`	`Map.forEach`
`Predicate<T>`	`T → boolean`	`test(t)`	`filter`, `removeIf`
`BiPredicate<T,U>`	`(T, U) → boolean`	`test(t, u)`	two-argument checks
`Function<T,R>`	`T → R`	`apply(t)`	`map`, key extractors
`BiFunction<T,U,R>`	`(T, U) → R`	`apply(t, u)`	`Map.merge`, `compute`
`UnaryOperator<T>`	`T → T`	`apply(t)`	`List.replaceAll`, transformations
`BinaryOperator<T>`	`(T, T) → T`	`apply(a, b)`	`reduce`, `merge` functions

Older JDK types are functional interfaces too: `Runnable` (`() → void`), `Callable<V>` (`() → V`, may throw), `Comparator<T>`. `UnaryOperator<T>` extends `Function<T, T>` and `BinaryOperator<T>` extends `BiFunction<T, T, T>`, so they inherit `apply`. Typical targets: `list.removeIf(...)` takes a `Predicate<? super E>`; `map.merge(k, 1, ...)` takes a `BiFunction<? super V, ? super V, ? extends V>` (commonly `Integer::sum`).

Primitive specialisations

Generics (Chapter 9) only work with objects, so a `Function<Integer, Integer>` **boxes** every `int` into an `Integer` (allocating outside the -128..127 `Integer` cache, §7.6). The primitive versions avoid that. Their names follow a pattern:

Pattern	Meaning	Examples
`IntX`, `LongX`, `DoubleX`	the **input** is that primitive	`IntPredicate`, `IntFunction<R>`, `IntConsumer`, `IntUnaryOperator`, `IntBinaryOperator`
`ToIntX`, `ToLongX`, `ToDoubleX`	the **output** is that primitive	`ToIntFunction<T>`, `ToDoubleBiFunction<T,U>`
`IntToLongFunction`, `DoubleToIntFunction`, …	primitive to primitive	
`XSupplier`	supplies a primitive	`IntSupplier` (`getAsInt`), `BooleanSupplier` (`getAsBoolean`)
`ObjIntConsumer<T>` etc.	object + primitive, no result	`(s, n) -> …`

The method names change accordingly: `applyAsInt`, `applyAsDouble`, `getAsInt`, `test`, `accept`. Only `int`, `long` and `double` get full sets (plus `BooleanSupplier`) — there is no `CharFunction` or `FloatPredicate`.

Code

```
// code/ch11/functional/BuiltInInterfaces.java (key parts)
Supplier<List<String>> newList = ArrayList::new;
Consumer<String> log = s -> System.out.println("log: " + s);
BiConsumer<String, Integer> put = (k, v) -> System.out.println(k + "=" + v);
Predicate<Order> big = o -> o.amount() > 100;
Function<Order, String> toId = Order::id;
BiFunction<Double, Double, Double> pow = Math::pow;
UnaryOperator<String> shout = s -> s.toUpperCase() + "!";
BinaryOperator<Integer> max = BinaryOperator.maxBy(Comparator.naturalOrder());

IntPredicate even = n -> n % 2 == 0;           IntUnaryOperator square = n -> n * n;
ToIntFunction<String> len = String::length;    IntFunction<String> stars = "*"::repeat;
ObjIntConsumer<String> repeatPrint = (s, n) -> System.out.println(s.repeat(n));

Function<Integer, Integer> boxedInc = x -> x + 1;   // boxes on every call
IntUnaryOperator primInc = x -> x + 1;              // no boxing
```

Output:

```
log: supplier made [x]
retries=3
big? true, longerThan("java",3)? true, id=A-1, 2^10=1024.0, HEY!, max=9
even(6)=true square(7)=49 add=5 len=5 stars=*** dice=4 ratio=0.25
ababab
20M calls: Function<Integer,Integer> 57 ms vs IntUnaryOperator 8 ms (machine-dependent) results equal=true
```

The last line is a rough single-run timing (Apple M4 Pro, JDK 26 — **machine-dependent**; the JIT compiler can sometimes remove boxing on its own). The point stands: in hot loops over numbers, pick the primitive interface.

When to write your own

- The shape doesn't exist (three parameters, or it throws a checked exception — §11.8).
- A domain name adds clarity and you'll attach default methods (`Discount.then` in §11.2).
- Annotate it with `@FunctionalInterface`.

TL;DR

Supplier (get), Consumer (accept), Predicate (test), Function (apply) plus `Bi` / `Unary` / `Binary` variants cover almost everything. Use `Int` / `Long` / `Double` versions for primitives to avoid boxing.

11.4 Method references

What it is

A **method reference** is shorthand for a lambda that only calls one existing method: `String::length` means `s -> s.length()`. It also needs a target functional interface. *Effective Java* Item 43: prefer them when clearer than the lambda — and keep the lambda when not.

The four kinds

Kind	Syntax	Equivalent lambda	Example
Static method	`Class::staticMethod`	`(args) -> Class.staticMethod(args)`	`Integer::parseInt`
Bound instance (specific object)	`obj::method`	`(args) -> obj.method(args)`	`System.out::println`, `prefix::concat`
Unbound instance (any object of a type)	`Class::instanceMethod`	`(obj, args) -> obj.method(args)`	`String::toUpperCase`, `String::startsWith`
Constructor	`Class::new`	`(args) -> new Class(args)`	`ArrayList::new`, `User::new`, `int[]::new`

The **unbound** form confuses people: the *first* parameter of the functional interface becomes the receiver. So `BiFunction<String, String, Boolean> f = String::startsWith` means `(s, p) -> s.startsWith(p)`, and `Comparator<String> c = String::compareToIgnoreCase` means `(a, b) -> a.compareToIgnoreCase(b)`.

Code

```java
// code/ch11/methodrefs/MethodRefDemo.java (key parts)
record User(String name, int age) {
    boolean isAdult() { return age >= 18; }
    static int byAge(User a, User b) { return Integer.compare(a.age, b.age); }
}
Function<String, Integer> parse = Integer::parseInt;          // static
Comparator<User> ageOrder = User::byAge;                      // static, shape (User, User) -> int
Function<String, String> addPrefix = prefix::concat;          // bound
Consumer<Object> printer = System.out::println;               // bound
Function<String, String> upper = String::toUpperCase;         // unbound
Predicate<User> adult = User::isAdult;                        // unbound
BiFunction<String, String, Boolean> startsWith = String::startsWith;   // unbound, 2 params
Supplier<List<User>> listMaker = ArrayList::new;              // constructor
BiFunction<String, Integer, User> makeUser = User::new;       // constructor with 2 args
IntFunction<int[]> arrayMaker = int[]::new;                   // array constructor

StringBuilder sb = new StringBuilder("v1");
Supplier<String> snap = sb::toString;                         // receiver sb fixed now
sb.append("+v2");
```

Output:

```
[User[name=Zed, age=17], User[name=Kim, age=25], User[name=Amy, age=34]]
43 ID-7 OK adult(Zed)=false startsWith=true array length=5
4 5
bound receiver is the same object: v1+v2
Amy Kim Zed
```

Notes:

- `parse.apply("42") + 1` printed `43`: the `Integer` result was unboxed and added before string concatenation started.
- `String::length` fits both `ToIntFunction<String>` and `Function<String, Integer>` — the target type decides.
- A **bound** reference evaluates its receiver expression **once**, when the reference is created (throwing `NullPointerException` right then if it's `null`). It holds that *object*, so later mutations are visible (`v1+v2`), but reassigning the variable afterwards would not change the reference.
- Overloads can make a reference ambiguous (e.g. `Integer::toString` fits both static `toString(int)` and instance `toString()` for a `Function<Integer, String>`) — use a lambda then.

TL;DR

Four kinds: `Class::static`, `obj::method` (bound), `Class::method` (unbound: first argument is the receiver), `Class::new`. Use when it reads better than the lambda.

11.5 Variable capture and "effectively final"

What it is

A lambda can use variables from the surrounding code — **capturing**; a function carrying captured variables is a **closure**. Java's rule: a captured **local variable or parameter** must be **final or effectively final** — never reassigned after initialisation (whether or not it says `final`).

Why the rule exists

1. Locals live on the method's **stack frame**, which disappears when the method returns, but a lambda can outlive it (stored, or passed to another thread, Chapter 14). So Java **copies the value** into the lambda object.
2. If the variable could change, copy and original would silently disagree; forbidding reassignment keeps the copy correct.
3. It also rules out **data races** on locals (several threads touching one variable, at least one writing, uncoordinated).

Fields are different. Instance fields are reached through the captured `this` (static fields directly), so lambdas may read and modify them freely — which is exactly why shared **mutable** fields are an easy source of thread-safety bugs.

Code

```
// code/ch11/capture/CaptureDemo.java (key parts)
int threshold = 10;                          // effectively final
Predicate<Integer> above = x -> x > threshold;
// threshold = 11;                           // would make the lambda above a compile error

for (String s : List.of("a", "b", "c")) suppliers.add(() -> s.toUpperCase());   // new 's' each iteration: fine
for (int i = 0; i < 3; i++) { int copy = i; byIndex.add(() -> copy * 10); }    // i is reassigned: copy it

AtomicInteger count = new AtomicInteger();                  // mutable holder: legal but a smell
List.of("x", "yy", "zzz").forEach(w -> count.addAndGet(w.length()));
int better = List.of("x", "yy", "zzz").stream().mapToInt(String::length).sum();   // return a result instead

Runnable lambda = () -> System.out.println("lambda this -> " + this.name);        // enclosing instance
Runnable anon = new Runnable() { public void run() { System.out.println(this.getClass().getName()); } };
Runnable inc = () -> counterField++;                        // fields may be modified
```

Output:

```
above(12)=true
ABC
0 10 20
holder count=6 vs stream sum=6
lambda this      -> CaptureDemo instance
anonymous this   -> ch11.capture.CaptureDemo$1
field modified from lambda: 2
```

Common mistakes

- Capturing the classic `for (int i…)` index — it's reassigned by `i++`. Copy to a local or use `IntStream.range(0, n).forEach(i -> …)`. (The enhanced-for variable is fine: each iteration declares a fresh, never-reassigned variable.)
- Working around the rule with one-element arrays (`int[] c = {0}; list.forEach(x -> c[0]++)`) or an `AtomicInteger` just to accumulate. It compiles, but re-creates shared mutable state that breaks if the code later runs in parallel. Compute a result instead (`sum` , `count` , `collect`).
- Assuming a captured *object* is frozen — only the variable is effectively final; the object can still be mutated.

TL;DR

Captured locals must be effectively final (values are copied into the lambda). Fields aren't restricted. Don't smuggle mutation through arrays or atomics — return results instead.

11.6 Composing functions

What it is

Composition builds a new function from existing ones, like joining pipes. The functional interfaces ship default and static methods for it, so behaviour is assembled from small, tested, named pieces.

Interface	Composition methods
`Function`	`f.andThen(g)` = g(f(x)) (f first); `f.compose(g)` = f(g(x)) (g first); `Function.identity()` returns its input unchanged (handy in `toMap(identity(), …)`)
`Predicate`	`and`, `or`, `negate`, `Predicate.not(p)` (11), `Predicate.isEqual(x)`
`Consumer` / `BiConsumer`	`andThen` — run one after the other
`BiFunction`	`andThen(Function)`
`BinaryOperator`	`minBy(comparator)`, `maxBy(comparator)`
`UnaryOperator`	`identity()`
`Comparator`	`comparing`, `thenComparing`, `reversed`, `nullsFirst` (§10.8)

Code

```
// code/ch11/composition/CompositionDemo.java (key parts)
Function<Integer, Integer> plus2 = x -> x + 2, times3 = x -> x * 3;
plus2.andThen(times3).apply(5);     // (5 + 2) * 3
plus2.compose(times3).apply(5);     // 5 * 3 + 2

UnaryOperator<String> trim = String::strip, lower = String::toLowerCase;
UnaryOperator<String> collapse = s -> s.replaceAll("\\s+", " ");
Function<String, String> clean = trim.andThen(lower).andThen(collapse);

Predicate<String> nonBlank = Predicate.not(String::isBlank);
Predicate<String> shortWord = s -> s.length() <= 4, startsJ = s -> s.startsWith("J");
filter(words, nonBlank.and(shortWord));   filter(words, startsJ.or(shortWord));
filter(words, startsJ.negate().and(nonBlank));   filter(words, Predicate.isEqual("Go"));

Consumer<String> audit = s -> System.out.print("[audit " + s + "] ");
audit.andThen(send).accept("msg-1");

Function<Double, UnaryOperator<Double>> taxAdder = rate -> price -> price * (1 + rate);   // function factory
UnaryOperator<Double> vat20 = taxAdder.apply(0.20);
```

Output:

```
plus2.andThen(times3)(5) = 21, plus2.compose(times3)(5) = 17
identity = same
clean -> [hello world]
nonBlank&short: [Java, Go, JVM]  startsJ|short: [Java, , Jakarta, Go,   , JVM]  !startsJ: [Go]  isEqual(Go): [Go]
[audit msg-1] [send msg-1]
area.andThen(format) = 12 m2, minBy length = fig
vat20(50.0) = 60.0
length, then reverse alpha: [a, cc, bb, aa, ddd]
```

Notes:

- `and` / `or` **short-circuit** like `&&` / `||`: the second predicate isn't evaluated when the first decides.

- `startsJ.or(shortWord)` kept `""` and `" "` because they are short — combine with `nonBlank` when that matters.
- `Predicate.not(String::isBlank)` exists because you can't write `String::isBlank.negate()` — a method reference has no type until it's assigned.
- `trim.andThen(lower)` returns a `Function`, not a `UnaryOperator` (the default method is inherited from `Function`).

Real-world examples

Validation rules (`Predicate<Order>` combined per market), request-processing **middleware** (a chain of `UnaryOperator<Request>` adding logging or auth before the handler), text-normalisation pipelines for search indexing, configurable pricing rules (function factories).

TL;DR

Build big behaviour from small functions: `andThen` / `compose` for functions, `and` / `or` / `negate` / `not` for predicates, `andThen` for consumers, and functions that return functions for configurable logic.

11.7 Optional

What it is

`Optional<T>` (Java 8) is a container holding either one non-null value or nothing. It is meant for **method return types** where "no result" is normal — `findById`, `max`, `findFirst` — so the signature tells the caller to handle absence.

Why it matters

A returned `null` says nothing in the signature, and a forgotten check causes a `NullPointerException` far from the cause. `Optional` makes absence explicit and replaces nested `if (x != null)` chains with fluent operations.

The API

Group	Methods
Create	`Optional.of(v)` (NPE if `v` is null), `Optional.ofNullable(v)`, `Optional.empty()`
Test	`isPresent()`, `isEmpty()` (11)
Transform	`map(f)` (wraps the result; a `null` result becomes empty), `flatMap(f)` (when `f` already returns an `Optional`), `filter(p)`
Extract	`orElse(default)`, `orElseGet(supplier)`, `orElseThrow()` (10), `orElseThrow(exceptionSupplier)`, `get()` (throws if empty)
Act	`ifPresent(consumer)`, `ifPresentOrElse(consumer, runnable)` (9)
Combine	`or(() -> otherOptional)` (9), `stream()` (9) → 0 or 1 elements
Primitive versions	`OptionalInt`, `OptionalLong`, `OptionalDouble` (`getAsInt()` …)

Code

```java
// code/ch11/optional/OptionalDemo.java (key parts)
record Address(String city, String zip) {}
record Customer(String name, Address address, String email) {
    Optional<Address> addressOpt() { return Optional.ofNullable(address); }
    Optional<String> emailOpt() { return Optional.ofNullable(email); }
}
static Optional<Customer> findById(int id) { return Optional.ofNullable(DB.get(id)); }

String city = findById(1).flatMap(Customer::addressOpt).map(Address::city).orElse("no city");

findById(1).map(Customer::name).orElse(expensiveDefault());            // default computed ANYWAY
findById(1).map(Customer::name).orElseGet(OptionalDemo::expensiveDefault); // computed only if empty

findById(9).orElseThrow(() -> new IllegalArgumentException("customer 9 not found"));
findById(2).flatMap(Customer::emailOpt).ifPresentOrElse(e -> ..., () -> System.out.println("Bo has no email ->
send SMS"));
findById(2).flatMap(Customer::emailOpt).or(() -> Optional.of("sms:+000"));
List.of(1, 2, 9).stream().map(OptionalDemo::findById).flatMap(Optional::stream)...   // drop empties
```

Output:

```
Optional[hi] Optional.empty Optional.empty isPresent=true isEmpty=true
Optional.of(null) -> NPE
cities: Lisbon, no city, no city
gmail? false
orElse on present:    (computing default) Ana
orElseGet on present: Ana
orElseThrow() -> No value present
orElseThrow(supplier) -> customer 9 not found
Bo has no email -> send SMS
fallback Optional via or(): sms:+000
all known emails: [ana@example.com]
OptionalInt max=3, empty average=OptionalDouble.empty
anti-pattern isPresent+get: Ana
better: Ana
```

Read `findById(id).flatMap(Customer::addressOpt).map(Address::city).orElse("no city")` as: find the customer → get the address (itself optional, so `flatMap` avoids `Optional<Optional<Address>>`) → take the city → else "no city". Three "nothing" points, zero `if`s.

The `orElse` line proves that **`orElse`'s argument is always evaluated** — it's an ordinary method argument. Use `orElseGet` when the default is expensive or has side effects (§11.8).

Right uses and wrong uses

✅ Do	❌ Don't
Return `Optional<T>` from a method when "no result" is normal	Return `null` from a method declared to return `Optional`
Chain `map` / `flatMap` / `filter` and finish with `orElse*` / `orElseThrow` / `ifPresent*`	`if (opt.isPresent()) opt.get()` — a verbose null check
Use `orElseThrow()` (10) when absence is a bug — its name says it can throw	Call `get()` without a check (`orElseThrow()` is the same thing with an honest name)
Use `OptionalInt` / `OptionalDouble` for primitives	`Optional<Integer>` in numeric code
Return an **empty collection** for "no items"	`Optional<List<T>>` — an empty list already means "none"
Use nullable fields internally and expose `Optional` from getters if helpful	`Optional` **fields** (not `Serializable` ; extra object per field)
Overload or use a default for optional inputs	`Optional` **parameters** (callers must wrap; they can still pass `null`)
Use it as a return value	Use it as a map key/value or in collections

These follow *Effective Java* Item 55 ("return optionals judiciously") and the JDK designers' intent: `Optional` is primarily for return types. It is also a *value-based* class (instances are pure values with no meaningful identity) — don't compare with `==` or synchronise on them (Chapter 14).

In the interview

> **30-second answer:** "Optional is a return-type container for 'maybe a value'. It makes absence explicit in the signature and lets me chain map, flatMap and filter and end with orElse, orElseGet or orElseThrow instead of null checks. I don't use it for fields, parameters or collections, I never return null from an Optional method, and I use orElseGet when the default is expensive because orElse evaluates its argument eagerly."

- *Follow-up:* `map` *vs* `flatMap` *?* → Use `flatMap` when the function itself returns an `Optional`, to avoid nesting.
- *Follow-up:* `Optional.of` *vs* `ofNullable` *?* → `of` throws NPE on null (use when null is a bug); `ofNullable` returns empty.
- *Follow-up: Is* `Optional` *serializable?* → No — one reason not to use it for fields.

TL;DR

`Optional` = explicit "maybe" for return values. Transform with `map` / `flatMap` / `filter`, end with `orElse` / `orElseGet` / `orElseThrow` / `ifPresentOrElse`. Not for fields, parameters or collections; never `null` itself.

11.8 Functional style in everyday Java

What it is

Functional programming (FP) builds programs from **functions** that compute results from inputs, avoiding shared mutable state. Java is not a pure functional language, but several FP ideas make Java code simpler and safer. Key terms:

Term	Meaning
Pure function	Result depends only on its arguments; no side effects (no I/O, no mutation of outside state). Same input → same output, every time.
Side effect	Anything a function does besides returning a value: printing, writing a field, changing an argument, I/O.
Immutability	Data that can't change after creation (records, `List.of`, `String`).
Higher-order function	Takes a function as a parameter and/or returns one (`retryUntil`, `memoize`, `Comparator.comparing`).
Currying	Turning `f(a, b)` into `a -> b -> …`, so you can fix `a` first (**partial application**).
Laziness	Delaying a computation until its result is needed (`Supplier`, streams in Chapter 12).
Memoisation	Caching a pure function's results by input.
Declarative	Describe *what* result you want (`filter`, `map`) rather than *how* to loop — the style Chapter 12 is built on.

Code

```
// code/ch11/fp/FunctionalStyle.java (key parts)
static int total = 0;
static int addToTotal(int x) { total += x; return total; }    // impure: result depends on history
static int add(int a, int b) { return a + b; }                // pure

static <T> List<T> retryUntil(Supplier<T> attempt, Predicate<T> ok, int max) { ... }   // higher-order

Function<Integer, Function<Integer, Integer>> curriedAdd = a -> b -> a + b;
Function<Integer, Integer> add10 = curriedAdd.apply(10);      // partial application

static void logIf(boolean enabled, Supplier<String> message) { if (enabled) System.out.println(message.get()); }
logIf(debug, () -> "state dump " + "x".repeat(1_000_000).length());   // string never built when debug=false

static <T, R> Function<T, R> memoize(Function<T, R> f) {
    Map<T, R> cache = new HashMap<>();
    return x -> { R r = cache.get(x); if (r == null) { r = f.apply(x); cache.put(x, r); } return r; };
}

Map<String, DoubleUnaryOperator> shipping = Map.of(           // strategy pattern as data
        "standard", w -> 4.99, "express", w -> 9.99 + 1.5 * w, "free", w -> 0.0);

@FunctionalInterface interface ThrowingFunction<T, R> { R apply(T t) throws Exception; }
static <T, R> Function<T, R> unchecked(ThrowingFunction<T, R> f) {
    return t -> {
        try { return f.apply(t); }
        catch (RuntimeException e) { throw e; }
        catch (Exception e) { throw new RuntimeException(e); }
    };
}
```

Output:

```
impure twice: 5, 10   pure twice: 5, 5
retryUntil six: [2, 5, 6]
add10(5)=15
lazy log: supplier not called because debug=false
memoized: 5 calls, underlying function ran 2 times
standard(2kg)=4.99  express(2kg)=12.99  free(2kg)=0.00
app.properties=ok
wrapped: java.io.IOException: no such file: missing.properties
original [10, 20, 30] untouched; new [9, 18, 27]
```

What each idea buys you:

- **Pure functions** are trivially testable (no setup, no mocks), safe to cache and safe to run in parallel.
- **Higher-order functions** remove duplicated control flow: the retry loop is written once; callers pass *what* to try and *when* it succeeded.
- **Laziness via** `Supplier` is how logging frameworks avoid building expensive messages (SLF4J's fluent API and `java.util.logging.Logger.fine(Supplier<String>)` accept suppliers).
- **Strategies as lambdas** replace a hierarchy of one-method classes; see the Strategy pattern in LLD Chapter 10.
- **Immutability**: transform into a new list instead of editing in place; the original stays valid for anyone else holding it.

Memoisation note. The obvious `cache.computeIfAbsent(x, f)` is wrong when `f` is *recursive* (e.g. memoised Fibonacci calling back into the same map): a `HashMap` must not be modified by its own mapping function and throws `ConcurrentModificationException` (since Java 9); `ConcurrentHashMap` can **deadlock** or throw `IllegalStateException`. The get-then-put version above is safe single-threaded.

Checked exceptions (Chapter 8): `java.util.function` methods don't declare `throws`, so a lambda calling `Files.readString` (throws `IOException`) won't compile as a `Function<Path, String>`. Options: handle it inside the lambda, wrap and rethrow unchecked as above (`UncheckedIOException` is the JDK's own choice for I/O), or define a throwing interface. Never silently swallow it.

When not to use functional style

- The loop is simpler: a `for` with early `return` or several interacting variables is often clearer than a clever chain.
- Deeply nested lambdas (a function returning a function returning a function) — name intermediate pieces.
- Performance-critical inner loops where boxing or **megamorphic** calls (one call site seeing many implementing classes, which stops the JIT inlining it) matter — measure (Chapter 16).
- Stack traces through lambdas include **synthetic** (compiler-generated) frames like `lambda$main$0`; debugging a long chain is harder than a named method.

Real-world examples

Spring's `RestClient` / `WebClient` builders and Security configuration **DSLs** (domain-specific languages: mini-APIs that read like a small language) take lambdas; `CompletableFuture` chains functions over async results (Chapter 15); UI listeners and callbacks; retry/backoff helpers (Resilience4j decorates `Supplier`s and `Function`s); JUnit's `assertThrows(Class, Executable)` (Chapter 18).

In the interview

> **30-second answer:** "In Java I use functional ideas where they reduce bugs: pure functions and immutable data for anything shared or tested, higher-order functions to remove repeated control flow, Suppliers for laziness, and lambdas as strategies. I stop when a plain loop is clearer, and I'm careful with checked exceptions and mutable captured state."

- *Follow-up: Is Java a functional language?* → No — it's object-oriented with functional features (functions as storable, passable values via functional interfaces, lambdas, streams, records, pattern matching). Functions are objects implementing interfaces, not a separate function type.
- *Follow-up: What's a pure function and why care?* → Output depends only on input, no side effects (so `x -> { System.out.println(x); return x; }` is impure); easy to test, cache, and parallelise.

TL;DR

Pure functions + immutable data + higher-order functions = code that's easier to test and parallelise. Use `Supplier` for laziness, lambdas for strategies, wrap checked exceptions deliberately, and fall back to loops when they're clearer.

Chapter TL;DR

Topic	Remember
Why	Pass behaviour as a value; lambdas replace anonymous classes (EJ Item 42)
Functional interface	Exactly one abstract method; `@FunctionalInterface`; lambda's type comes from the target
Syntax	`x -> expr`, `(a, b) -> { …; return r; }`, `(var a, var b)` (11), `_` for unused (22)
Built-ins	`Supplier.get`, `Consumer.accept`, `Predicate.test`, `Function.apply`, `Bi*`, `UnaryOperator`, `BinaryOperator`; `IntX` / `ToIntX` to avoid boxing
Method refs	`Class::static`, `obj::method`, `Class::method` (first arg is receiver), `Class::new`
Capture	Locals must be effectively final (value copied); fields free; `this` = enclosing instance
Composition	`andThen` / `compose` / `identity`, `and` / `or` / `negate` / `Predicate.not`, functions returning functions
Optional	Return type only; `map` / `flatMap` / `filter` → `orElse` / `orElseGet` (lazy)/ `orElseThrow`; no fields/params/collections; never null
FP style	Pure, immutable, higher-order, lazy; wrap checked exceptions; loops when clearer

Where next: put all of this to work on collections with Chapter 12 — Streams. Exam traps: OCP Chapter 8. Python equivalents (lambdas, closures, `functools`): Python Chapter 7.